

PLATFORM ARCHITECTURE & RELATIONAL ENGINEERING

Database Schema & Relational Topology

Exhaustive Data Model, Entity Interconnections, Invariants & Lifecycle State Machines

ENGINE & STORAGE	ACTIVE ENTITIES	TENANCY MODEL	DOCUMENT VERSION
PostgreSQL 15+ / PGlite (ACID)	15 Relational Tables / 9 Enums	Multi-Tenant Scoped (Strict FK)	v1.2.0 (Post-P0/P3 Hardening)

1. Architectural Guarantees & System Invariants

The database schema is constructed by first-principles to provide an uncompromising transactional backbone for enterprise carpooling. Every operational query, corridor match, seat allocation, and administrative action conforms to four foundation invariants:

1. Strict Multi-Tenant Boundary Isolation

Every operational table maintains an explicit, non-nullable `organization_id UUID` referencing `organizations.id`. All unique and performance indexes are compound-keyed with `organization_id`, ensuring absolute cross-tenant data isolation and preventing data leakage across corporate entities.

2. Clean Domain & Asset Separation

Identity (`users`) is decoupled from physical assets (`vehicles`), scheduled journeys (`rides`), corridor vectors (`ride_routes`), and booking tenders (`ride_requests`). Commuter pickup/drop coordinates are never stored directly on the ride; routes are never opaque strings.

3. Concurrency Safety & Seat Integrity

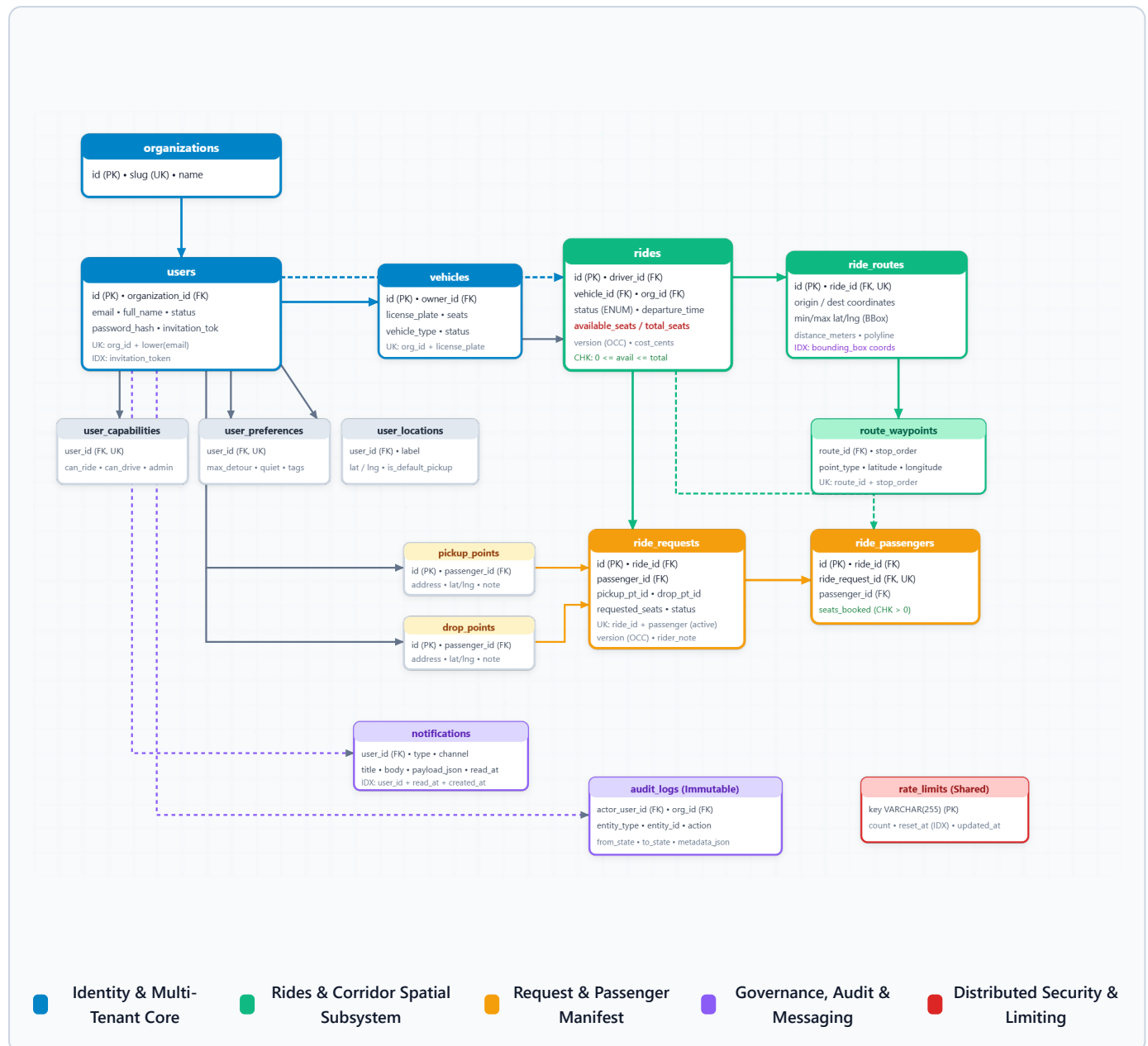
Seats cannot be oversold under concurrent load. Enforced via database-level check constraints (`chk_available_seats_non_negative` : `available_seats >= 0` and `available_seats <= total_seats_offered`), combined with PostgreSQL row-level pessimistic locking (`FOR UPDATE`) in transactional workflows.

4. Geospatial Corridor Optimization

Spatial routes are structured with 4D bounding boxes (`min/max latitude/longitude`) and ordered waypoint nodes (`route_waypoints`). Search queries execute sub-millisecond bounding box containment tests before corridor polygon calculations, eliminating expensive table scans.

2. Canonical Entity-Relationship Topology (ERD)

The diagram below depicts all 15 tables in the system, color-coded by domain cluster. Lines illustrate foreign key relationships and cardinalities across tenant boundaries, driver scheduling, and passenger matching.



3. Domain Subsystems & Lifecycle Interconnections

3.1 Identity, Tenancy & Vehicle Ownership

The tenant root is `organizations`. Every employee record in `users` is bounded by its parent organization. Capabilities (`can_ride`, `can_drive`, `is_org_admin`) are maintained in a 1-to-1 extension table `user_capabilities` to keep the core identity table compact and audit-friendly. Vehicles are owned by users (`vehicles.owner_id`) and must be verified before drivers can schedule rides with them.

3.2 Scheduled Rides, Corridors & Bounding Box Spatial Indexing

When a driver posts a ride, three tightly coupled entities are populated:

- `rides`: Tracks temporal schedules, status, and seat counters (`available_seats` and `total_seats_offered`).
- `ride_routes`: Contains the departure and arrival coordinates, total route duration, distance, polyline, and geographic bounding box (`min_latitude`, `max_latitude`, `min_longitude`, `max_longitude`).
- `route_waypoints`: Normalized intermediate stops along the corridor with cumulative `estimated_arrival_offset_seconds`.

During search, the query uses the indexed bounding box coordinates to quickly eliminate 99% of non-matching rides before executing corridor detour evaluations.

3.3 Request Negotiation, Optimistic Concurrency & Pessimistic Seat Locking

Passengers submit requests via `ride_requests`, referencing their chosen `pickup_points` and `drop_points`. When a driver accepts a request, concurrency control prevents race conditions:

Atomic Acceptance & Booking Workflow (PostgreSQL Transaction)

1. `SELECT * FROM ride_requests WHERE id = $1 FOR UPDATE;` (Row-level exclusive lock on request)
2. `SELECT * FROM rides WHERE id = $2 FOR UPDATE;` (Row-level exclusive lock on ride)
3. Verify `available_seats >= requested_seats` and driver ownership.
4. Decrement `available_seats = available_seats - requested_seats` on `rides`.
5. Transition request status to `ACCEPTED`.
6. Insert confirmed booking record into `ride_passengers`.
7. Commit transaction. If available seats violate constraints, PostgreSQL raises an invariant error and aborts.

4. Exhaustive Data Dictionary (All 15 Tables)

4.1 organizations — Tenant Partitioning Root

Represents the corporate tenant. All platform operations are strictly isolated by organization.

Column	Type	Attributes	Description
<code>id</code>	UUID	PK default <code>gen_random_uuid()</code>	Unique organization identifier
<code>name</code>	VARCHAR(255)	NOT NULL	Corporate legal or trade name
<code>slug</code>	VARCHAR(100)	UK NOT NULL	URL and subdomain slug
<code>allowed_email_domains</code>	TEXT[]	NOT NULL	Whitelist of permitted corporate email domains
<code>settings</code>	JSONB	NOT NULL DEFAULT '{}'	Custom tenant configuration policies
<code>is_active</code>	BOOLEAN	NOT NULL DEFAULT TRUE	Tenant platform access flag
<code>created_at</code> / <code>updated_at</code>	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	Creation and last modification timestamp

4.2 users — Identity & Authentication Ledger

Corporate employee profiles, password hashes, and identity provider linkages.

Column	Type	Attributes	Description
<code>id</code>	UUID	PK default <code>gen_random_uuid()</code>	Unique user identifier
<code>organization_id</code>	UUID	FK NOT NULL → <code>organizations(id)</code>	Tenant association (ON DELETE RESTRICT)
<code>email</code>	VARCHAR(255)	NOT NULL	Corporate email address
<code>full_name</code>	VARCHAR(255)	NOT NULL	Employee display name
<code>phone_number</code>	VARCHAR(50)	NULLABLE	Contact phone number
<code>status</code>	user_status	NOT NULL DEFAULT 'ACTIVE'	<code>ACTIVE</code> , <code>INACTIVE</code> , <code>SUSPENDED</code> , <code>PENDING_VERIFICATION</code>
<code>work_department</code>	VARCHAR(100)	NULLABLE	Employee department
<code>work_location</code>	VARCHAR(255)	NULLABLE	Assigned campus or office building
<code>password_hash</code>	VARCHAR(255)	NULLABLE	Scrypt salt:hash string for native authentication
<code>invitation_token</code>	VARCHAR(100)	IDX NULLABLE	One-time token for account activation
<code>invitation_token_expires_at</code>	TIMESTAMPTZ	NULLABLE	Expiration timestamp for invitation token

Indexes & Constraints: `uq_users_org_lower_email` UNIQUE (`organization_id`, `lower(email)`); `idx_users_org_status` INDEX (`organization_id`, `status`); `idx_users_invitation_token` INDEX (`invitation_token`).

4.3 user_capabilities — Permissions & Verification

Column	Type	Attributes	Description
<code>id</code>	UUID	PK default gen_random_uuid()	Capability record ID
<code>user_id</code>	UUID	FK UK → users(id)	User reference (ON DELETE CASCADE)
<code>organization_id</code>	UUID	FK → organizations(id)	Tenant association
<code>can_ride</code>	BOOLEAN	NOT NULL DEFAULT TRUE	Authorized to book rides as passenger
<code>can_drive</code>	BOOLEAN	NOT NULL DEFAULT FALSE	Authorized to offer rides as driver
<code>is_org_admin</code>	BOOLEAN	NOT NULL DEFAULT FALSE	Administrative rights for tenant organization
<code>driver_verified_at</code>	TIMESTAMPTZ	NULLABLE	Timestamp when driver license was validated

4.4 user_preferences — Commuter Personalization

Column	Type	Attributes	Description
<code>id</code>	UUID	PK default gen_random_uuid()	Preference record ID
<code>user_id</code>	UUID	FK UK → users(id)	User reference (ON DELETE CASCADE)
<code>organization_id</code>	UUID	FK → organizations(id)	Tenant association
<code>free_text_preferences</code>	TEXT	NULLABLE	Natural language commuting notes
<code>tags</code>	TEXT[]	DEFAULT '{}'	Categorical preference tags (e.g. non-smoker)
<code>max_detour_minutes</code>	INTEGER	DEFAULT 15	Maximum acceptable detour minutes
<code>quiet_ride</code>	BOOLEAN	DEFAULT FALSE	Preference for silent commute

4.5 vehicles — Driver Fleet & Asset Registry

Column	Type	Attributes	Description
<code>id</code>	UUID	PK default <code>gen_random_uuid()</code>	Unique vehicle identifier
<code>owner_id</code>	UUID	FK NOT NULL → <code>users(id)</code>	Driver / vehicle owner (ON DELETE RESTRICT)
<code>organization_id</code>	UUID	FK NOT NULL → <code>organizations(id)</code>	Tenant association
<code>make</code> / <code>model</code> / <code>year</code>	VARCHAR / SMALLINT	NOT NULL	Vehicle manufacturer details
<code>license_plate</code>	VARCHAR(20)	NOT NULL	Official vehicle registration plate
<code>total_seats</code>	SMALLINT	NOT NULL	Manufacturer physical seat count
<code>max_passenger_capacity</code>	SMALLINT	NOT NULL DEFAULT 4	Maximum seats available for carpooling
<code>status</code>	vehicle_status	NOT NULL DEFAULT 'ACTIVE'	<code>ACTIVE</code> , <code>INACTIVE</code> , <code>PENDING_INSPECTION</code>

Indexes & Constraints: `uq_vehicles_org_plate` UNIQUE (`organization_id` , `license_plate`); `idx_vehicles_owner` INDEX (`owner_id`).

4.6 rides — Scheduled Carpool Journeys

The operational core of the platform. Hosts scheduled rides and enforces atomic seat accounting.

Column	Type	Attributes	Description
<code>id</code>	UUID	PK default <code>gen_random_uuid()</code>	Unique ride identifier
<code>organization_id</code>	UUID	FK NOT NULL → <code>organizations(id)</code>	Tenant boundary
<code>driver_id</code>	UUID	FK NOT NULL → <code>users(id)</code>	Ride host / driver
<code>vehicle_id</code>	UUID	FK NOT NULL → <code>vehicles(id)</code>	Vehicle deployed for ride
<code>status</code>	ride_status	NOT NULL DEFAULT 'DRAFT'	<code>DRAFT</code> , <code>SCHEDULED</code> , <code>IN_PROGRESS</code> , <code>COMPLETED</code> , <code>CANCELLED</code>
<code>departure_time</code>	TIMESTAMPTZ	NOT NULL	Planned origin departure datetime
<code>arrival_time_estimated</code>	TIMESTAMPTZ	NOT NULL	Estimated destination arrival datetime
<code>total_seats_offered</code>	SMALLINT	CHK > 0 NOT NULL	Initial seats opened by driver
<code>available_seats</code>	SMALLINT	CHK >= 0 NOT NULL	Remaining open bookable seats
<code>cost_per_seat_cents</code>	INTEGER	NOT NULL DEFAULT 0	Shared expense reimbursement in cents
<code>version</code>	INTEGER	NOT NULL DEFAULT 1	OCC version counter for optimistic locking

Check Constraints: `chk_available_seats_non_negative` (`available_seats >= 0`), `chk_available_seats_le_total` (`available_seats <= total_seats_offered`), `chk_total_seats_positive` (`total_seats_offered > 0`).

Indexes: `idx_rides_org_departure` (`organization_id`, `departure_time`, `status`); `idx_rides_driver` (`driver_id`, `departure_time`).

4.7 ride_routes — Spatial Corridor Definition

Column	Type	Attributes	Description
<code>id</code>	UUID	PK default <code>gen_random_uuid()</code>	Route identifier
<code>ride_id</code>	UUID	FK UK → <code>rides(id)</code>	Associated ride (ON DELETE CASCADE)
<code>origin_address</code> / <code>destination_address</code>	TEXT	NOT NULL	Human-readable addresses
<code>origin_latitude</code> / <code>origin_longitude</code>	NUMERIC(10,7)	NOT NULL	Departure coordinates
<code>destination_latitude</code> / <code>destination_longitude</code>	NUMERIC(10,7)	NOT NULL	Arrival coordinates
<code>total_distance_meters</code> / <code>total_duration_seconds</code>	INTEGER	NOT NULL	Route metrics from routing engine
<code>min_latitude</code> , <code>max_latitude</code> , <code>min_longitude</code> , <code>max_longitude</code>	NUMERIC(10,7)	NOT NULL	Geographic bounding box for search queries
<code>encoded_polyline</code>	TEXT	NULLABLE	Google Encoded Polyline algorithm string

Indexes: `idx_ride_routes_bbox` INDEX (`min_latitude` , `max_latitude` , `min_longitude` , `max_longitude`).

4.8 route_waypoints — Corridor Transit Stops

Column	Type	Attributes	Description
<code>id</code>	UUID	PK default <code>gen_random_uuid()</code>	Waypoint identifier
<code>route_id</code>	UUID	FK NOT NULL → <code>ride_routes(id)</code>	Route association (ON DELETE CASCADE)
<code>stop_order</code>	SMALLINT	NOT NULL	Sequence order (0 = Origin, N = Destination)
<code>point_type</code>	waypoint_type	NOT NULL	<code>ORIGIN</code> , <code>CORRIDOR</code> , <code>DESTINATION</code> , <code>PASSENGER_STOP</code>
<code>latitude</code> / <code>longitude</code>	NUMERIC(10,7)	NOT NULL	Waypoint coordinates
<code>estimated_arrival_offset_seconds</code>	INTEGER	NOT NULL DEFAULT 0	Seconds from departure time to this stop

Indexes: `uq_route_waypoints_order` UNIQUE (`route_id` , `stop_order`); `idx_route_waypoints_coords` INDEX (`latitude` , `longitude`).

4.9 pickup_points & drop_points — Commuter Boarding Locations

Column	Type	Attributes	Description
<code>id</code>	UUID	PK default <code>gen_random_uuid()</code>	Point identifier
<code>passenger_id</code>	UUID	FK NOT NULL → <code>users(id)</code>	Commuter requesting pickup or drop
<code>organization_id</code>	UUID	FK NOT NULL → <code>organizations(id)</code>	Tenant association
<code>address_text</code>	TEXT	NOT NULL	Physical street or campus address
<code>latitude</code> / <code>longitude</code>	NUMERIC(10,7)	NOT NULL	Precise GPS location
<code>landmark_note</code>	TEXT	NULLABLE	Visual pickup cues (e.g. "Gate 3 near cafeteria")

4.10 ride_requests — Booking Negotiations

Column	Type	Attributes	Description
<code>id</code>	UUID	PK default <code>gen_random_uuid()</code>	Request identifier
<code>ride_id</code>	UUID	FK NOT NULL → <code>rides(id)</code>	Target ride journey
<code>passenger_id</code>	UUID	FK NOT NULL → <code>users(id)</code>	Commuter requesting seat
<code>pickup_point_id</code> / <code>drop_point_id</code>	UUID	FK NOT NULL	References <code>pickup_points</code> / <code>drop_points</code>
<code>requested_seats</code>	SMALLINT	NOT NULL DEFAULT 1	Number of seats requested
<code>status</code>	<code>ride_request_status</code>	NOT NULL DEFAULT 'PENDING'	<code>PENDING</code> , <code>ACCEPTED</code> , <code>REJECTED</code> , <code>CANCELLED</code> , <code>EXPIRED</code>
<code>rider_note</code>	TEXT	NULLABLE	Note to host driver
<code>version</code>	INTEGER	NOT NULL DEFAULT 1	OCC version counter

Indexes: `uq_ride_requests_active_passenger` UNIQUE (`ride_id` , `passenger_id`) WHERE `status` IN ('PENDING', 'ACCEPTED'); `idx_ride_requests_ride` (`ride_id` , `status`); `idx_ride_requests_passenger` (`passenger_id` , `status`).

4.11 ride_passengers — Confirmed Manifest

Column	Type	Attributes	Description
<code>id</code>	UUID	PK default <code>gen_random_uuid()</code>	Manifest entry ID
<code>ride_id</code>	UUID	FK NOT NULL → <code>rides(id)</code>	Target ride
<code>ride_request_id</code>	UUID	FK UK → <code>ride_requests(id)</code>	Originating request (1:1)
<code>passenger_id</code>	UUID	FK NOT NULL → <code>users(id)</code>	Confirmed passenger
<code>seats_booked</code>	SMALLINT	CHK > 0 NOT NULL	Guaranteed reserved seats
<code>cost_charged_cents</code>	INTEGER	NOT NULL DEFAULT 0	Final fare charged to commuter

Constraints: `chk_seats_booked_positive` (`seats_booked > 0`); `uq_ride_passengers_ride_user` UNIQUE (`ride_id`, `passenger_id`).

4.12 notifications — User Messaging Log

Column	Type	Attributes	Description
<code>id</code>	UUID	PK default gen_random_uuid()	Notification record ID
<code>user_id</code>	UUID	FK NOT NULL → users(id)	Recipient (ON DELETE CASCADE)
<code>type</code>	notification_type	NOT NULL	<code>RIDE_REQUESTED</code> , <code>REQUEST_ACCEPTED</code> , <code>REQUEST_REJECTED</code> , etc.
<code>channel</code>	notification_channel	NOT NULL DEFAULT 'IN_APP'	<code>IN_APP</code> , <code>EMAIL</code> , <code>SMS</code> , <code>PUSH</code>
<code>title</code> / <code>body</code>	VARCHAR / TEXT	NOT NULL	Message content delivered to user
<code>payload_json</code>	JSONB	NOT NULL DEFAULT '{}'	Action deep-links and ride metadata
<code>read_at</code>	TIMESTAMPTZ	NULLABLE	Read receipt timestamp

Indexes: `idx_notifications_user` INDEX (`user_id` , `read_at` , `created_at`).

4.13 audit_logs — Tamper-Evident Compliance Trail

Immutable ledger of all state transitions, booking authorizations, and administrative actions.

Column	Type	Attributes	Description
<code>id</code>	UUID	PK default gen_random_uuid()	Log entry ID
<code>organization_id</code>	UUID	FK NOT NULL → organizations(id)	Tenant association
<code>actor_user_id</code>	UUID	FK NULLABLE → users(id)	Actor responsible (ON DELETE SET NULL)
<code>entity_type</code>	VARCHAR(50)	NOT NULL	Target domain table (e.g. <code>rides</code> , <code>ride_requests</code>)
<code>entity_id</code>	UUID	NOT NULL	Target primary key UUID
<code>action</code>	audit_action	NOT NULL	<code>CREATE</code> , <code>UPDATE</code> , <code>STATE_TRANSITION</code> , <code>DELETE</code> , <code>CANCEL</code>
<code>from_state</code> / <code>to_state</code>	VARCHAR(50)	NULLABLE	State transition values if applicable
<code>metadata_json</code>	JSONB	NOT NULL DEFAULT '{}'	Structured snapshot of altered attributes
<code>ip_address</code> / <code>user_agent</code>	VARCHAR / TEXT	NULLABLE	Client network forensics

Indexes: `idx_audit_logs_org` INDEX (`organization_id` , `created_at` , `entity_type`).

4.14 rate_limits — Distributed Security Sliding Window

Shared rate-limiting store preventing credential stuffing and API abuse across instances.

Column	Type	Attributes	Description
<code>key</code>	VARCHAR(255)	PK	Bucketed limit key (e.g. <code>login:127.0.0.1</code>)
<code>count</code>	INTEGER	NOT NULL DEFAULT 1	Cumulative request counter within window
<code>reset_at</code>	TIMESTAMPTZ	IDX NOT NULL	Window expiration timestamp
<code>created_at</code> / <code>updated_at</code>	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	Creation and update timestamps

Security Invariant: In staging and production environments, the rate limiter fails closed if this table cannot be queried, ensuring zero bypass during outages.

5. State Machine Lifecycles & Transition Rules

5.1 Ride Lifecycle (`rides.status`)

From	To	Trigger & Side Effects
DRAFT	SCHEDULED	Driver publishes ride; seats are made discoverable in search.
SCHEDULED	IN_PROGRESS	Departure time arrived; driver starts trip. No new bookings permitted.
IN_PROGRESS	COMPLETED	Driver completes trip; manifests finalized; audit logged.
SCHEDULED	CANCELLED	Driver cancels; all confirmed passengers refunded; notifications dispatched.

5.2 Ride Request Lifecycle (`ride\_requests.status`)

From	To	Trigger & Side Effects
PENDING	ACCEPTED	Driver approves; <code>available_seats</code> decremented; passenger added to manifest.
PENDING	REJECTED	Driver declines; reason recorded; seats untouched.
PENDING	EXPIRED	Auto-expired by cron worker 30m before departure.
PENDING/ACCEPTED	CANCELLED	Rider cancels; if accepted, <code>available_seats</code> atomically incremented.
ACCEPTED	COMPLETED	Ride marked completed; passenger journey closed.